Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
speed@DESKTOP-P4G181S: ~
wsl: Unknown key 'wsl2.autoMemoryReclaim' in C:\Users\speed\.wslconfig:3
speed@DESKTOP-P4G181S:~$ nano task1.c
speed@DESKTOP-P4G181S:~$ gcc task1.c -o task1
./task1
=== Interactive Loop Program ===
Type commands below. Type 'exit' to quit.

myshell> Hello
You entered: Hello
myshell> Tejaswin Amara
You entered: Tejaswin Amara
myshell> exit
Exiting shell. Goodbye!
speed@DESKTOP-P4G181S:~$
```

```
speed@DESKTOP-P4G181S: ~
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {
    char input[BUFFER_SIZE];

    printf("=== Interactive Loop Program ===\n");
    printf("Type commands below. Type 'exit' to quit.\n\n");

    // 1. Main Loop
    while (1) {
        // 2. Display Prompt
        printf("myshell> ");
        fflush(stdout);

        // 3. Read User Input
        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\nExiting...\n");
            break;
        }

        // Remove the newline character '\n' from the end
        input[strcspn(input, "\n")] = '\0';

        // Ignore empty input (if user just presses Enter)
        if (strlen(input) == 0) {
            continue;
        }

        // 4. Handle Exit Condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell. Goodbye!\n");
            break;
        }

        // 5. Echo/Process command
        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: open(“ filename.txt”, modes, permissons)

Example: open(“First_Prgm.txt”, O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	**Purpose**
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

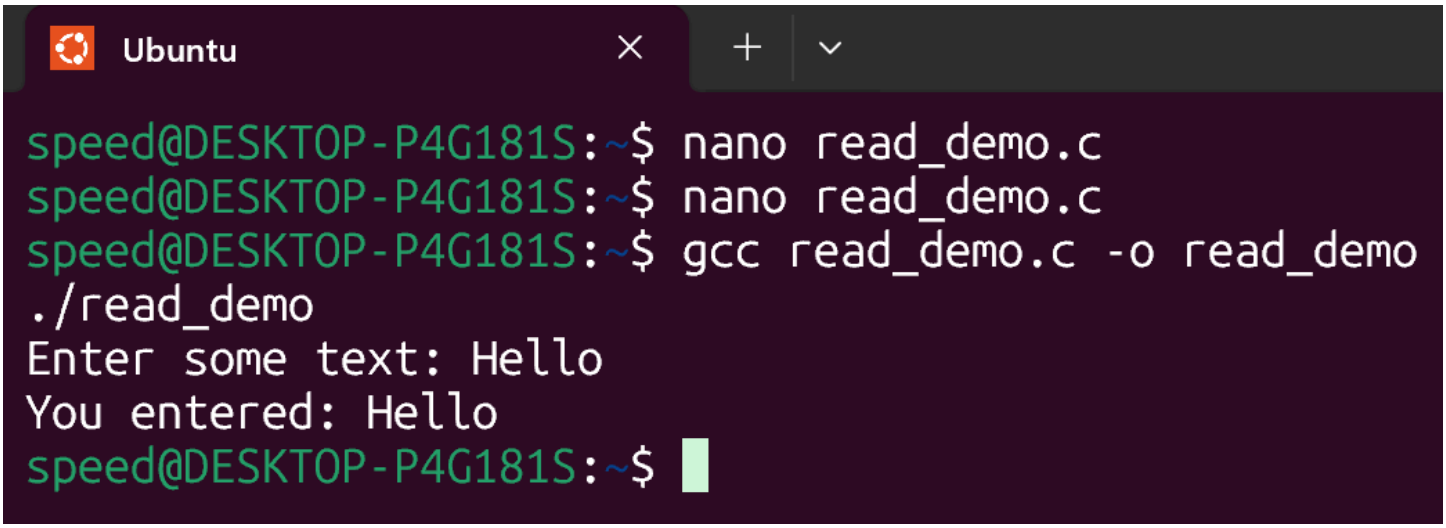
Example: read(fd, buffer, 100)

```
#include <stdio.h>
#include <unistd.h>
int main()
{
   char buffer[100];
   int n;
   printf("Enter some text: ");
   n = read(0, buffer, sizeof(buffer) - 1);
   buffer[n] = '\0';   // Null-terminate the string
```

```
    printf("You entered: %s", buffer);
    return 0;
}
```

write()
Syntax: write(int fd, const void *buf, size_t count);
Example: write(fd, message, 11);

```
#include <unistd.h>
int main()
{
    write(1, "Hello World\n", 12);
    return 0;
}
```



Ubuntu

GNU nano 8.7.1

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    char buffer[100];
    int n;

    printf("Enter some text: ");
    fflush(stdout);

    // Read from standard input (file descriptor 0)
    n = read(0, buffer, sizeof(buffer) - 1);
    if (n > 0) {
        buffer[n] = '\0'; // Null-terminate string
        printf("You entered: %s", buffer);
    }

    return 0;
}
```